Kunal Dodechani Roll No. 025 Batch – T12 Subject: ASD & DevOps

Experiment 4

Aim: To understand Continuous Integration, install Jenkins, and build CI/CD pipelines using Declarative and Scripted syntax.

LO mapped: LO

LO:

Theory:

Continuous Integration

Continuous Integration is a software development practice that promotes collaboration, early issue detection, and automated testing to enhance the quality and efficiency of software development. **What is Jenkins?**

Jenkins is an industry-standard, open-source automation server designed to facilitate Continuous Integration (CI) and Continuous Delivery/Deployment (CD) in modern software engineering. It serves as the central orchestration engine for modern DevOps practice, replacing manual build and deployment steps with automated, repeatable workflows.

Key Functional Areas:

- Software Lifecycle Automation: Jenkins automates critical development stages—including code compilation, static analysis, unit testing, artifact generation, environment provisioning, and production deployment.
- Early Issue Detection (CI): Every time developers commit code changes to a shared repository (e.g., GitHub or GitLab), Jenkins can automatically trigger a build and execute test suites. This provides immediate feedback on integration breaks or bugs early in the cycle.
- Extensible Architecture & Plugins: With an ecosystem of over 1,800 community-contributed plugins, Jenkins integrates seamlessly with nearly every tool in the software stack—including version control systems (Git, SVN), build tools (Maven, Gradle), container platforms (Docker, Kubernetes), and cloud infrastructure (AWS, Azure, GCP).
- Pipeline as Code: Jenkins allows teams to define complex, multi-stage workflows as code using Jenkinsfiles (written in Groovy). This ensures build logic is version-controlled, easily audit-able, and consistent across development, staging, and production environments.

What is a Jenkins Pipeline?

A Jenkins Pipeline is a set of automated processes that help developers and DevOps teams build, test, and deploy code in a CI/CD environment. In simple terms, it is a workflow that defines how software moves from code to production in an automated and repeatable manner.

- Key Benefits of Jenkins Pipeline:

- Automates CI/CD processes: Streamlines build, test, and release cycles. ○ Better Visibility: Offers clear insight into the build process steps.
- Pipeline as Code: Allows version control of the build logic. ○ Supports Complex Workflows: Enables parallel execution, conditional logic, and modular design.

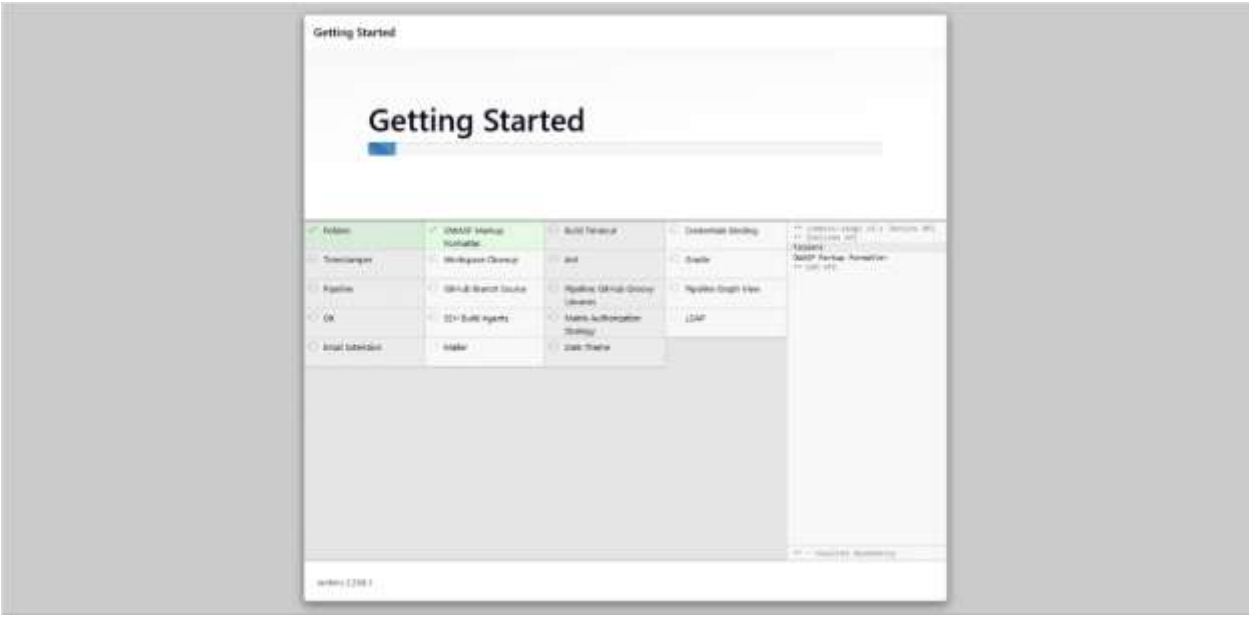
Types of Jenkins Pipelines:

2. Declarative Pipeline:
   - Uses a simpler, structured, and easy-to-read syntax. ○ Defined inside a pipeline {} block organized into stages and steps.
   - Provides better validation and simpler error handling in the Jenkins UI.
3. Scripted Pipeline:
   - Uses Groovy-based syntax for full flexibility and power. ○ Defined inside a node {} block where everything is written as code.
   - Ideal for complex workflows requiring advanced scripting and custom logic.

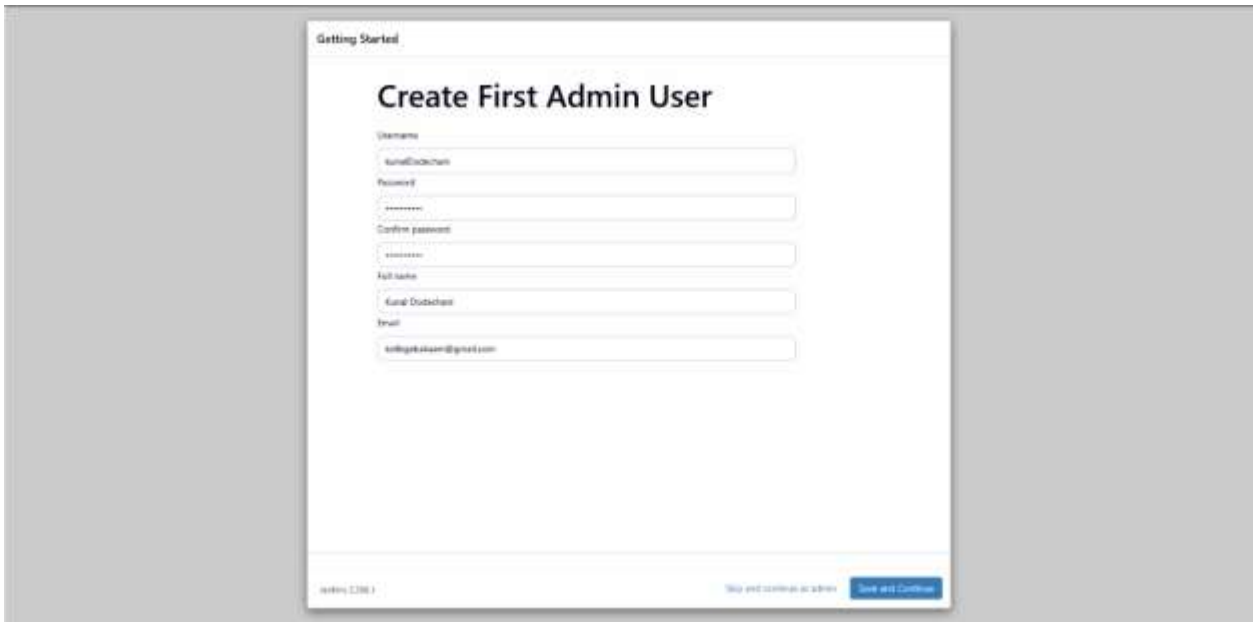
Steps Followed:

1. Java Verification: Verified that JDK is installed and configured on the system.
2. Download & Launch Jenkins: Downloaded jenkins.war and executed it via the terminal using java -jar jenkins.war.
3. Web Configuration: Accessed http://localhost:8080, entered the generated initialAdminPassword, installed suggested plugins, and configured the Admin account.
4. Pipeline Creation:
   - Created a Declarative Pipeline job executing a basic build step. ○ Created a Scripted Pipeline job using Groovy node syntax.
5. Execution: Ran both pipeline jobs and verified successful execution via the Console Output.

Output:

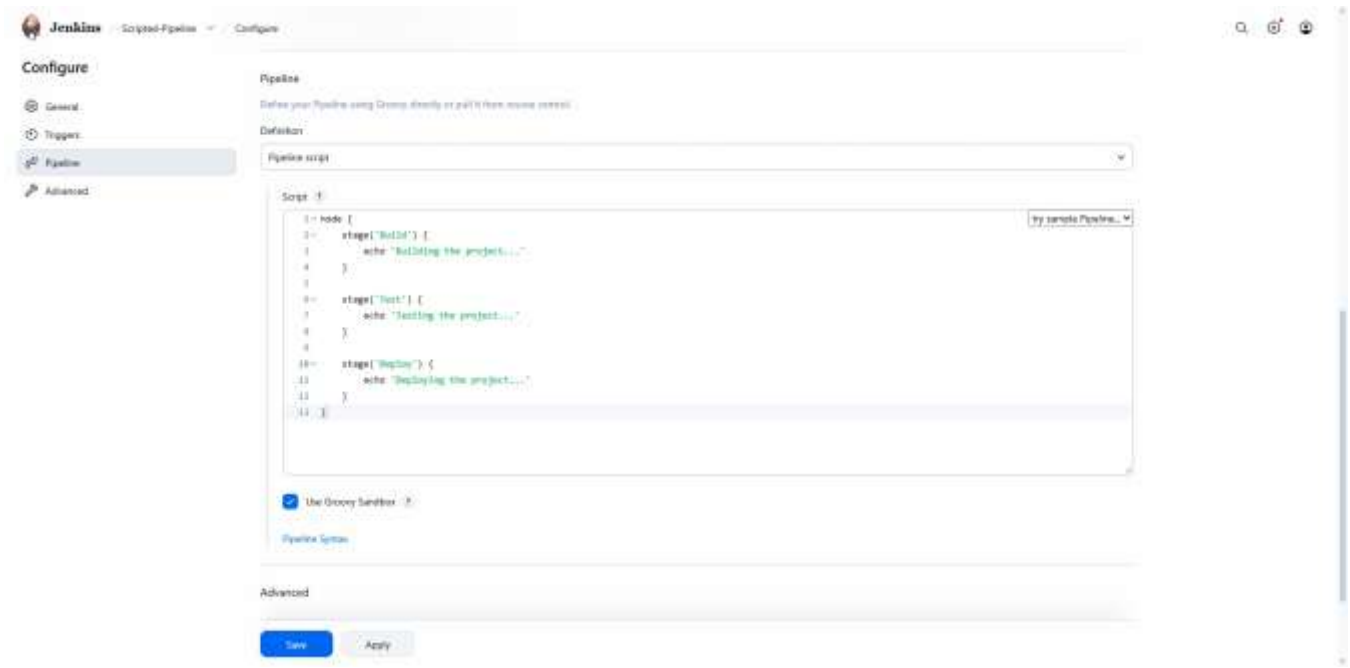

Getting Started
Getting Started

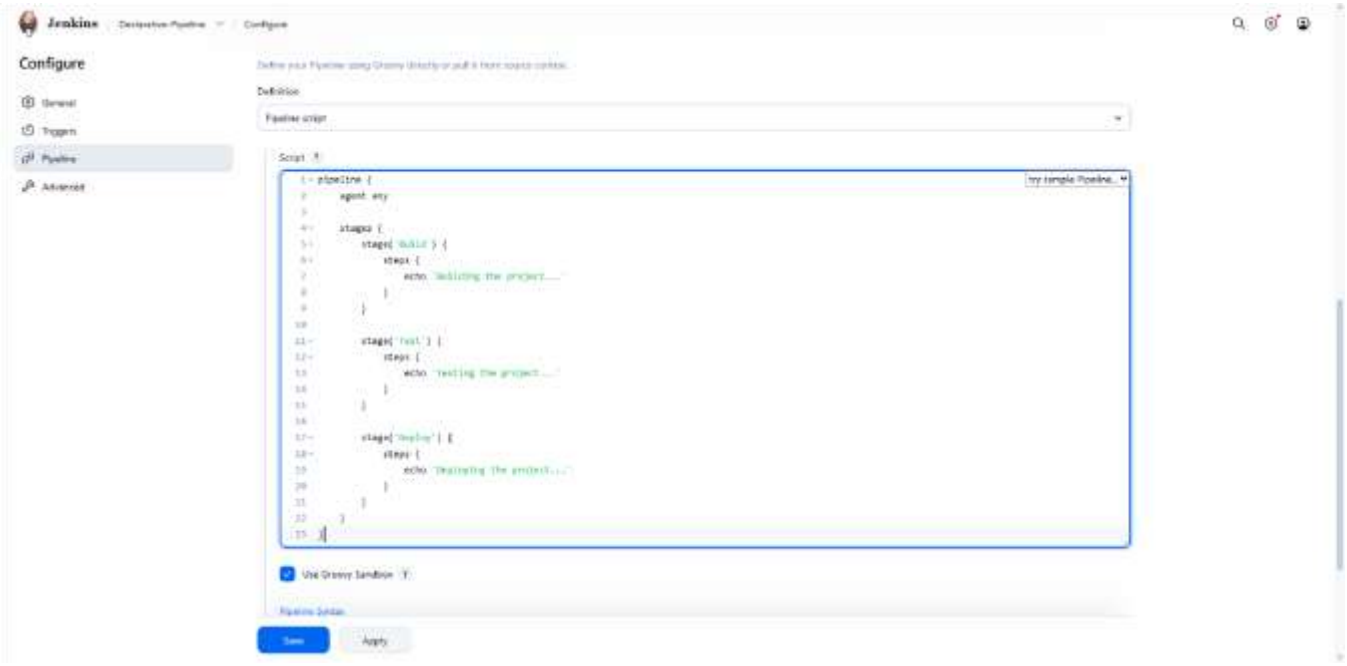

Getting Started
Customize Jenkins
Plugins extend Jenkins with additional features to support many different needs.
Install suggested plugins
Select plugins to install


Getting Started
Create First Admin User
Username
Password
Confirm password
Full name
Email


Getting Started
Instance Configuration


Getting Started
Jenkins is ready!
Your Jenkins setup is complete.


Jenkins
Configure
General
Triggers
Pipeline
Advanced
Pipeline
Definition
Pipeline script
Script
node {
stage('Build') {
echo 'Building the project...'
}
stage('Test') {
echo 'Testing the project...'
}
stage('Deploy') {
echo 'Deploying the project...'
}
}
Use Groovy Sandbox
Pipeline Syntax
Advanced
Save
Apply





Conclusion:

Through this experiment, we learned the fundamentals of Continuous Integration and successfully installed and configured Jenkins. Furthermore, we explored both Declarative and Scripted pipelines, demonstrating how Jenkins automates repeatable and efficient CI/CD workflows.

LO mapped successfully.